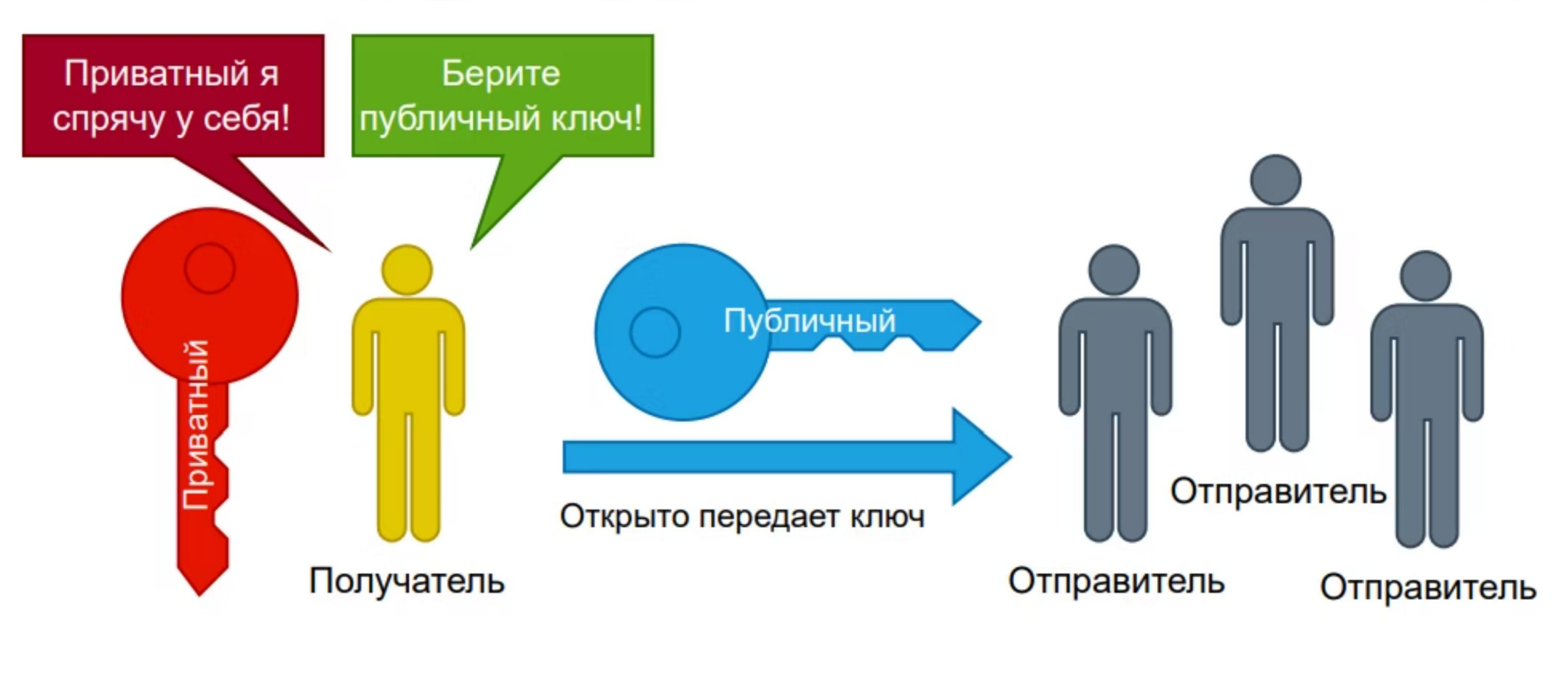
Асимметричное шифрование: секреты безопасности цифрового мира

1. Что такое асимметричное шифрование и как оно работает
2. Алгоритм RSA: математика, формулы, числовые примеры
3. Функция Эйлера и требования к ключам e и d
4. Шифрование текста: пример со словом ESIL
5. Гибридное шифрование RSA + AES
10. Контрольные вопросы и задания

ГЛАВА 1

Что такое асимметричное шифрование?

Революционный подход к защите информации, основанный на использовании пары математически связанных ключей



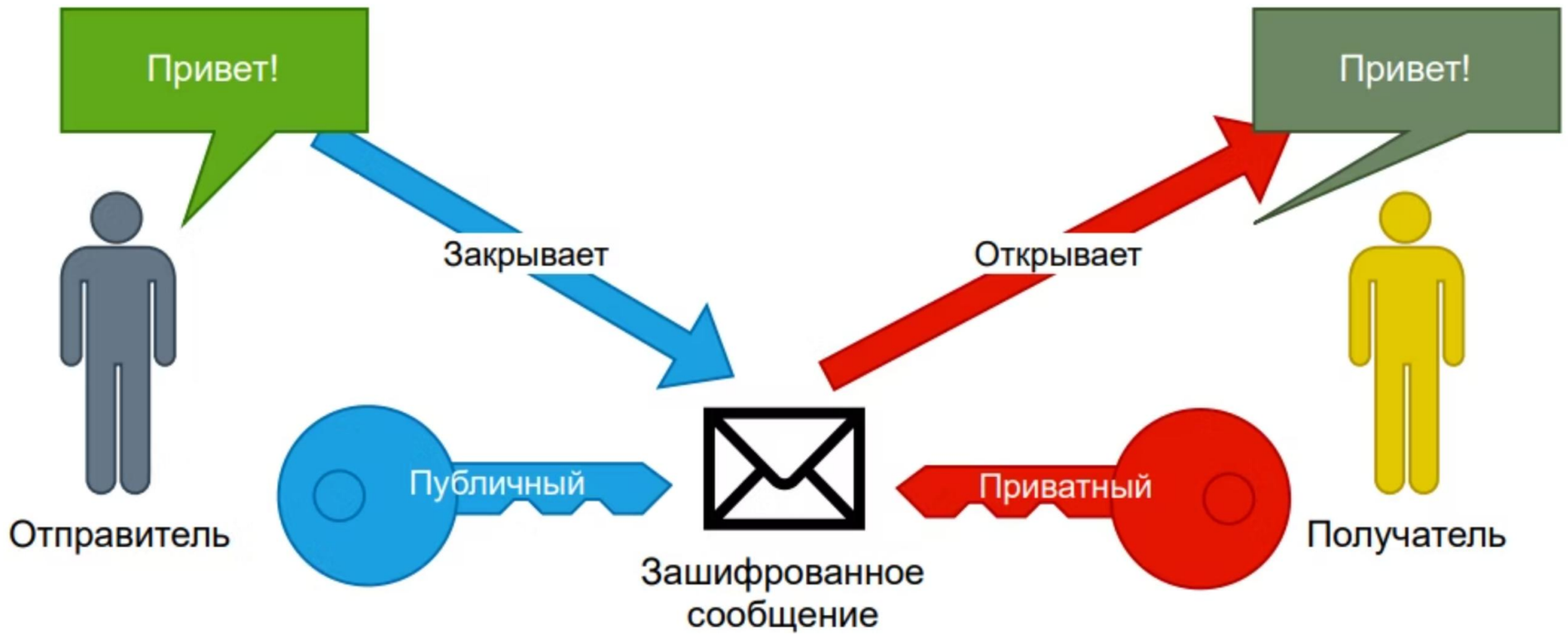
Два ключа — одна задача

Открытый ключ

Используется для шифрования данных. Можно свободно распространять и передавать любым пользователям без риска для безопасности

Закрытый ключ

Применяется для расшифровки информации. Должен храниться в строжайшем секрете и никогда не передаваться третьим лицам



Безопасность на основе пары ключей

Асимметричное шифрование использует элегантную математическую связь между двумя ключами: **то, что зашифровано одним ключом, может быть расшифровано только парным ему ключом.**

Эта уникальная особенность делает систему невероятно надёжной для защиты конфиденциальной информации.

Почему это важно?

Безопасная передача данных

Позволяет защищённо передавать конфиденциальную информацию даже по открытым и потенциально небезопасным каналам связи, включая интернет

Защита от перехвата

Даже если **злоумышленник перехватит зашифрованное сообщение**, без закрытого ключа расшифровать его практически невозможно

Гарантия подлинности

Технология позволяет не только защищать данные, но и **подтверждать подлинность отправителя** и **целостность полученной информации**

Как работает асимметричное шифрование?Математика безопасности

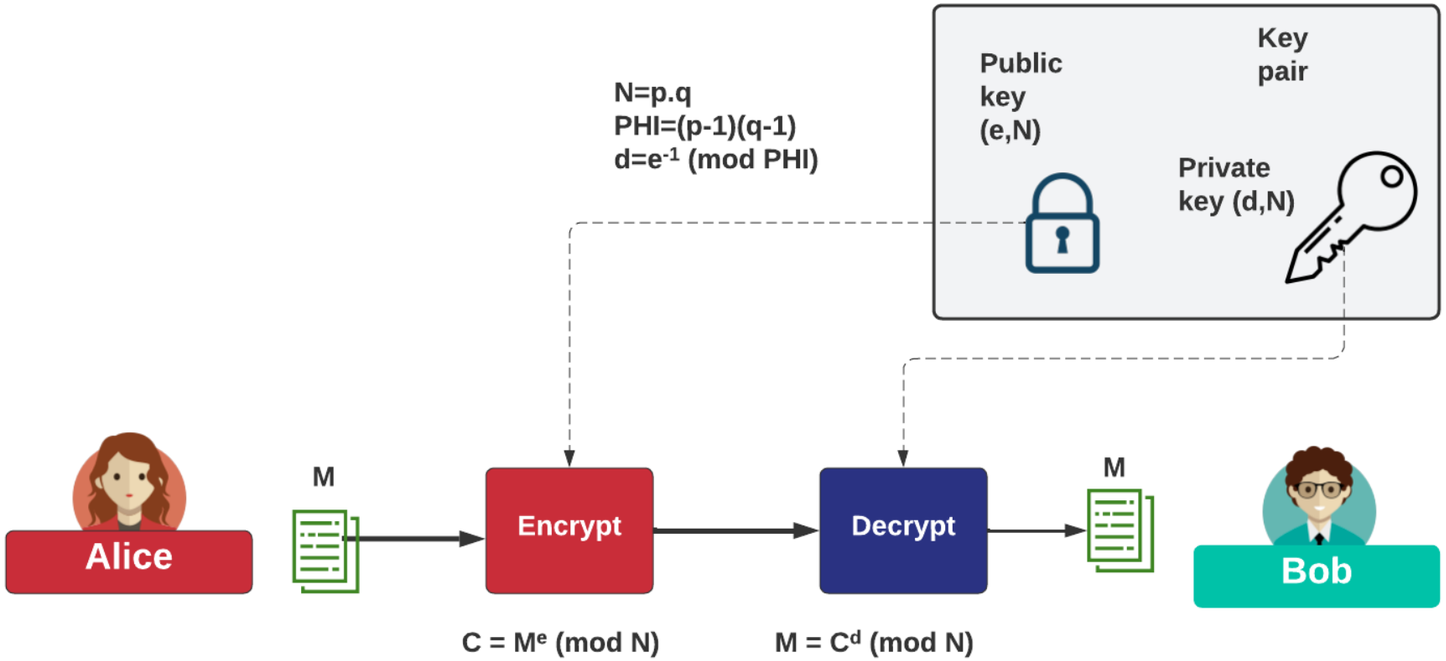
Фундамент криптографии

Асимметричное шифрование базируется на сложнейших математических задачах: **факторизация огромных чисел и вычисление дискретных логарифмов.**

> **Для разложения числа из 617 знаков на простые множители** современным компьютерам потребуются сотни лет непрерывных вычислений

Именно эта **вычислительная сложность обеспечивает надёжность защиты данных.**

Пример работы RSA



Генерация ключей

Создаётся пара математически связанн
открытый для шифрования и закрытый д
расшифровки

Шифрование сообщения

Отправитель использует открытый ключ получателя для защиты конфиденциальной информации

Расшифровка

Только владелец закрытого ключа может прочитать зашифрованное сообщение

Пример работы **RSA**

https://asecuritysite.com/rsa/

Генерация ключей

```
Выбираются два простых числа: p и q
n = p × q
ϕ(n)= (p−1)(q−1)
1 < e < ϕ(n), НОД(e, ϕ(n)) = 1
d × e ≡ 1 (mod ϕ(n))
Открытый ключ: (e, n)
Закрытый ключ: (d, n)
```

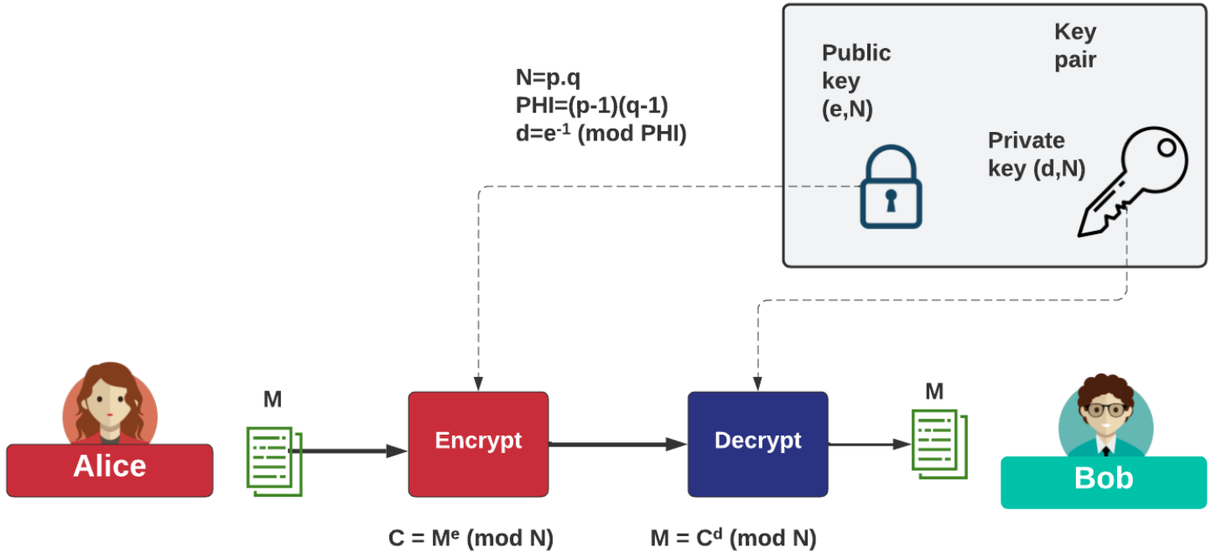
- **Обязательные условия:**
- **d × e ≡ 1 (mod φ(n))** — главное условие, d является обратным к e
- **1 < d < φ(n)** — d должно быть в этом диапазоне
- **d ≠ e** — иначе открытый и закрытый ключ совпадут
- **НОД(d, φ(n)) = 1** — следует автоматически из первого условия

https://python-academy.org/ru/handbook/math.gcd

Шифрование и расшифровка

```
Шифрование:
C = M^e mod n

Расшифровка:
M = C^d mod n
```



φ(n) — что это?

Считает количество целых чисел от 1 до n, которые взаимно просты с n (т.е. НОД с n равен 1).

Почему φ(n) = (p−1)(q−1) в RSA?Если n = p × q, где p и q — простые, то все числа от 1 до n взаимно просты с n, ***кроме*** кратных p и кратных q. Отсюда и формула.

Где применяется в RSA?
Она нужна для нахождения секретного ключа d:

d × e ≡ 1 (mod φ(n))

То есть d — это «обратный элемент» к e по модулю φ(n). Без знания φ(n) вычислить d практически невозможно — а φ(n) можно найти только зная p и q. Именно поэтому факторизация n обратно на p и q является главной защитой RSA.

В более широком смысле функция Эйлера используется в:

- Теореме Эйлера: **a^φ(n) ≡ 1 (mod n)** — именно на ней математически основана корректность RSA
- Теории чисел и других криптосистемах (например, Эль-Гамаль)

Обязательные условия для e:

- **$1 < e < \varphi(n)$** — e должно быть в этом диапазоне
- **НОД(e, φ(n)) = 1** — e взаимно просто с φ(n), это главное условие
- **e — простое число** (не обязательно, но на практике всегда так)

Неявные ограничения:

- **$e \neq p$ и $e \neq q$** — иначе НОД(e, φ(n)) ≠ 1, условие нарушится автоматически
- **$e \neq \varphi(n)$** — тогда НОД = φ(n) ≠ 1
- **$e \neq 1$** — тогда $C = M^1 \bmod n = M$, шифрования нет вообще
- **e не должно быть маленьким (e=3)** — уязвимость: если одно сообщение отправлено трём получателям с e=3, можно восстановить M китайской теоремой об остатках

Числовой пример **RSA**

Генерация ключей

```
p = 61, q = 53
n = 61 × 53 = 3233
ϕ(n) = 60 × 52 = 3120
e = 17  (НОД(17, 3120) = 1)
d = 2753 (17 × 2753 ≡ 1 mod 3120)

Открытый ключ: (17, 3233)
Закрытый ключ: (2753, 3233)
```

Шифрование

```
Сообщение: M = 65
C = 65^17 mod 3233
C = 2790
```

Расшифровка

```
C = 2790
M = 2790^2753 mod 3233
M = 65 ✓
```

Результат совпадает — шифрование работает корректно

RSA на Python

Простая реализация алгоритма **RSA** с вводом сообщения

```
def kee_e(f):
    for i in range(3, f, 2):
        a, b = i, f
        while b > 0:
            a, b = b, a % b
        # Если НОД равен 1, то число подходит
        if a == 1:
            return i
    return None

def kee_d(e, f):
    for i in range(2, f):
        if (i * e) % f == 1:
            return i

p = 7
q = 13
n = q * p
f_n = (p - 1) * (q - 1)
print(f_n)

e = kee_e(f_n)
print(e)

d = kee_d(e, f_n)
print(e, " ", d)

print("Open key", e, " ", n)
print("Close key", d, " ", n)

M = int(input("m="))
C = M**e % n
print(C)

C1 = C**d % n
print(C1)
```

Функции:

- `kee_e(f)` — находит e: перебирает нечётные числа, проверяет НОД через алгоритм Евклида
- `kee_d(e, f)` — находит d: перебором ищет обратный элемент к e по модулю φ(n)

Пример работы:

```
p=7, q=13
n = 91
ϕ(n) = 72
e = 5
d = 29
Open key:  5  91
Close key: 29 91
m= 10
C  = 10^5 mod 91 = 82
C1 = 82^29 mod 91 = 10 ✓
```

Справка. Операция ***mod*** вычисляет остаток от целочисленного деления двух чисел
Это соотношение означает, что результатом деления произведения чисел $\boldsymbol{e}$ и $\boldsymbol{d}$ на значение функции Эйлера должно быть число 1. Поэтому $\boldsymbol{d}$ можно рассчитать по формуле

$$\boldsymbol{d} = \frac{\boldsymbol{k} \cdot \boldsymbol{\varphi(n)} + 1}{\boldsymbol{e}}$$

altaev-aa.narod.ru

RSA-шифрование сообщения

RSA шифрование, задания

Шифрование текста: слово ESIL

Покажем пошаговое шифрование слова ESIL через RSA с ключами e=17, n=3233.

Перевод букв в ASCII-коды и шифрование

Буква	ASCII	Шифрование	Результат
E	69	69^17 mod 3233	2081
S	83	83^17 mod 3233	1521
I	73	73^17 mod 3233	2346
L	76	76^17 mod 3233	0065

Зашифрованное сообщение: [2081, 1521, 2346, 0065]

Расшифровка обратно

```
M = C^d mod n = C^2753 mod 3233

2081^2753 mod 3233 → E
1521^2753 mod 3233 → S
2346^2753 mod 3233 → I
0065^2753 mod 3233 → L

Результат: ESIL ✓
```

```
p = 61, q = 53
n = 61 × 53 = 3233
ϕ(n) = 60 × 52 = 3120
e = 17  (НОД(17, 3120) = 1)
d = 2753 (17 × 2753 ≡ 1 mod 3120)

Открытый ключ: (17, 3233)
Закрытый ключ: (2753, 3233)
```

Каждая буква шифруется независимо как отдельное число

https://www.youtube.com/shorts/zMmQ8vnyfEM

Гибридное шифрование: **RSA + AES**

Как асимметричное шифрование обеспечивает реальную безопасность в повседневных приложениях

Генерация сессионного ключа

Случайный симметричный ключ AES создается для каждой уникальной сессии связи.

Шифрование данных

Большие объемы данных (текст, файлы) шифруются быстрым алгоритмом AES с помощью этого сессионного ключа.

Шифрование сессионного ключа

Сам сессионный ключ AES шифруется открытым ключом получателя, используя RSA.

Передача

Получателю отправляются зашифрованные данные (AES) и зашифрованный сессионный ключ (RSA).

Расшифровка

Получатель расшифровывает сессионный ключ своим закрытым RSA-ключом, затем расшифровывает данные с его помощью.

Почему не только **RSA?**

- RSA медленный и ресурсоемкий для больших объемов данных.
- Ограничен размером блока (макс. ~256 байт для RSA-2048), не подходит для шифрования больших файлов.

Где применяется**?**

- HTTPS/TLS для безопасного просмотра веб-страниц.
- PGP-шифрование для защиты электронной почты.
- SSH для защищенного удаленного доступа.
- Современные мессенджеры (Signal, WhatsApp).

ГЛАВА 3

Гибридное шифрование: RSA + AES

Как RSA и AES работают вместе

Генерация AES-ключа

Случайный сессионный ключ

Шифрование данных

Файл или текст → AES → зашифрованные данные

Упаковка ключа

AES-ключ → RSA (открытый ключ) → зашифрованный ключ

Передача

Зашифрованные данные + зашифрованный ключ

Расшифровка

RSA (закрытый ключ) → AES-ключ → данные

AES — скорость

Шифрует гигабайты данных за миллисекунды

RSA — безопасность

Защищает ключ, но не данные напрямую

примерно в 1000–10 000 раз медленнее AES — именно поэтому им шифруют только маленький ключ (32 байта), а не сами данные.

RSA vs ECC: где что применяется сегодня

RSA не устарел — он просто используется там, где это оправдано

Область	RSA	ECC (ECDSA/ECDH)
HTTPS/TLS 1.2	✅ Широко используется	✅ Используется
HTTPS/TLS 1.3	⚠️ Вытесняется	✅ Предпочтительный
SSH	✅ RSA-2048/4096	✅ Ed25519 (быстрее)
Email (PGP/S-MIME)	✅ До сих пор популярен	✅ Поддерживается
ЭЦП Казахстана (НУЦ РК)	❌ Не используется	✅ ГОСТ 34.10-2015
Блокчейн / Bitcoin	❌	✅ secp256k1
JWT токены	✅ RS256	✅ ES256
Банковские системы	✅ RSA-2048	✅ Переходят на ECC
IoT устройства	⚠️ Тяжёлый для слабых CPU	✅ Идеален

Почему **RSA** ещё жив?

Огромная база legacy-систем, хорошо изучен, прост в реализации, поддерживается везде

Почему **ECC** побеждает?

Ключ в 8 раз короче при той же защите, в 10-40 раз быстрее, меньше трафика, идеален для мобильных и IoT

Основные алгоритмы асимметричного шифрования **RSA, DSA,** Эль-Гамаль, **ECDSA**

RSA

Самый популярный и широко используемый алгоритм. Базируется на сложности факторизации произведения двух больших простых чисел

DSA

Алгоритм цифровой подписи, основанный на вычислительной сложности дискретных логарифмов в конечных полях

Эль-Гамаль

Криптосистема на основе задачи дискретного логарифмирования, обеспечивающая высокий уровень безопасности

ECDSA

Использует эллиптические кривые для создания более компактных ключей. Активно применяется в блокчейн-технологиях и криптовалютах

Цифровые подписи

Цифровые подписи гарантируют подлинность отправителя и целостность документов, делая невозможным изменение данных после подписания.

1

Создание подписи

Документ подписывается закрытым ключом владельца, создавая уникальный цифровой отпечаток

2

Проверка подлинности

Любой пользователь может проверить подпись с помощью открытого ключа подписанта

Плюсы асимметричного шифрования

Безопасность передачи ключей

Открытый ключ можно свободно распространять по любым каналам связи без риска компрометации системы безопасности

Надёжная защита данных

Современные алгоритмы обеспечивают криптографическую стойкость, достаточную для защиты самой конфиденциальной информации

Цифровые подписи

Возможность создания неоспоримых электронных подписей для подтверждения подлинности документов и транзакций

Масштабируемость

Простота организации защищённого обмена данными между большим количеством пользователей без необходимости предварительного обмена секретными ключами

Минусы и ограничения

Основные недостатки

При всех преимуществах технология имеет определённые ограничения, которые важно учитывать

1. **Низкая скорость работы**

   Асимметричные алгоритмы работают значительно медленнее симметричных — в сотни и тысячи раз. Это делает их непрактичными для шифрования больших объёмов данных

2. **Сложность управления ключами**

   Для двусторонней защищённой связи каждой стороне необходимо сгенерировать собственную пару ключей и безопасно обменяться открытыми ключами

3. **Размер ключей**

   Для обеспечения достаточного уровня безопасности требуются ключи большой длины, что увеличивает вычислительную нагрузку и объём передаваемых данных

Ключевые различия https://habr.com/ru/articles/748226/

1

Симметричное шифрование

Использует один секретный ключ для шифрования и расшифровки данных. Требует безопасного обмена ключом между сторонами

- Высокая скорость работы
- Проблема безопасной передачи ключа
- Идеально для больших объёмов данных

2

Асимметричное шифрование

Применяет пару математически связанных ключей — открытый и закрытый. Обеспечивает повышенную безопасность

- Безопасный обмен ключами
- Медленная обработка данных
- Поддержка цифровых подписей

Когда что использовать?

Асимметричное шифрование

Идеально для:

- Безопасного обмена симметричными ключами
- Создания и проверки цифровых подписей
- Аутентификации пользователей и серверов
- Защиты небольших объёмов критически важных данных

Симметричное шифрование

Оптимально для:

- Быстрой обработки больших массивов информации
- Шифрования файлов и баз данных
- Защиты потокового видео и аудио
- Ситуаций, когда скорость критически важна

На практике часто используется гибридный подход: асимметричное шифрование для безопасной передачи ключа, симметричное — для шифрования основных данных

Где используется асимметричное шифрование?

SSL/TLS протоколы

Защита интернет-соединений (HTTPS). Каждый раз, когда вы видите замочек в адресной строке браузера, работает асимметричное шифрование

Электронные подписи

Юридически значимые цифровые подписи документов, сертификаты подлинности программного обеспечения и веб-сайтов

Криптовалюты и блокчейн

Генерация адресов кошельков, подтверждение транзакций, создание смарт-контрактов в Bitcoin, Ethereum и других блокчейн-системах

Защищённые коммуникации

Мессенджеры с end-to-end шифрованием (Signal, WhatsApp), защищённая электронная почта (PGP/GPG), VPN-соединения

Асимметричное шифрование стало фундаментом современной цифровой безопасности, незаметно защищая миллиарды транзакций и сообщений каждый день.https://encyclopedia.kaspersky.ru/glossary/asymmetric-encryption/

Контрольные вопросы

Проверьте свои знания по теме асимметричного шифрования

Теория

1. 1. В чём принципиальное отличие асимметричного шифрования от симметричного?
2. 2. Что такое открытый и закрытый ключ? Какой из них можно передавать по открытому каналу?
3. 3. Объясните назначение функции Эйлера φ(n) в алгоритме RSA.
4. 4. Почему RSA не используется для шифрования больших файлов напрямую?
5. 5. Что такое гибридное шифрование и зачем оно нужно?
6. 6. Чем ECDSA отличается от RSA? В чём преимущества ECC?

Практика

1. 7. Дано: p=5, q=11. Вычислите n, φ(n). Выберите e и найдите d.
2. 8. Зашифруйте букву "A" (ASCII=65) с помощью RSA при e=3, n=33.
3. 9. Почему в ECDSA нельзя использовать одно и то же случайное число k дважды?
4. 10. Какой алгоритм используется в ЭЦП Казахстана с 2024 года? Назовите стандарт.
5. 11. Сравните размер ключей RSA-2048 и ECC-256 при одинаковом уровне защиты.
6. 12. Объясните, почему задача дискретного логарифма на эллиптических кривых считается вычислительно сложной.

Задания для самостоятельной работы

Три уровня сложности

🟢 Базовый уровень

Задание 1: RSA вручную

Дано: p=7, q=13

- Вычислите n и φ(n)
- Выберите e (взаимно простое с φ(n))
- Найдите d через расширенный алгоритм Евклида
- Зашифруйте M=10, расшифруйте обратно

Задание 2: Ответьте письменно

Объясните своими словами: почему нельзя восстановить закрытый ключ, зная только открытый ключ RSA?

🟡 Средний уровень

Задание 3: Гибридное шифрование

Опишите пошагово, как браузер устанавливает HTTPS-соединение с сервером. Какую роль играет RSA или ECC? Что такое сессионный ключ?

КОНСПЕКТ Что такое протокол HTTPS и принципы его работы

Задание 4: Программирование

Напишите программу на Python, которая:

- Генерирует ключи RSA (p, q малые)
- Шифрует введённое слово побуквенно
- Расшифровывает обратно
- Выводит все промежуточные шаги

📝 Аудиторное задание.

Выполните самостоятельно на бумаге

Задание (RSA вручную): Зашифровать и расшифровать **третью букву вашего имени** с помощью алгоритма **RSA**

Дано: $p = 3$, $q = 11$

Шаг 1. Вычислите $n = p \times q$

Шаг 2. Вычислите $\varphi(n) = (p-1)(q-1)$

Шаг 3. Выберите $e = 7$ Проверьте: НОД$(7, \varphi(n)) = ?$

Шаг 4. Найдите d такое, что: $d \times 7 \equiv 1 \pmod{\varphi(n)}$ Подсказка: перебор $d = 1, 2, 3...$

Шаг 5. Найдите **порядковый номер третьей буквы вашего имени** в латинском алфавите (A=1, B=2, ..., Z=26) и зашифруйте её:

$C = M^e \bmod n = ?$

Шаг 6. Расшифруйте C обратно: $M = C^d \bmod n = ?$

M = **порядковый номер третьей буквы вашего имени** в латинском алфавите

Должен получиться исходный номер ✓

💡 Для простоты используйте латинскую транслитерацию имени. Например: Asel → третья буква 'e' → ASCII = 101. Проверьте: $101^7 \bmod 33 = ?$, затем результат$^3 \bmod 33 = 101$ ✓